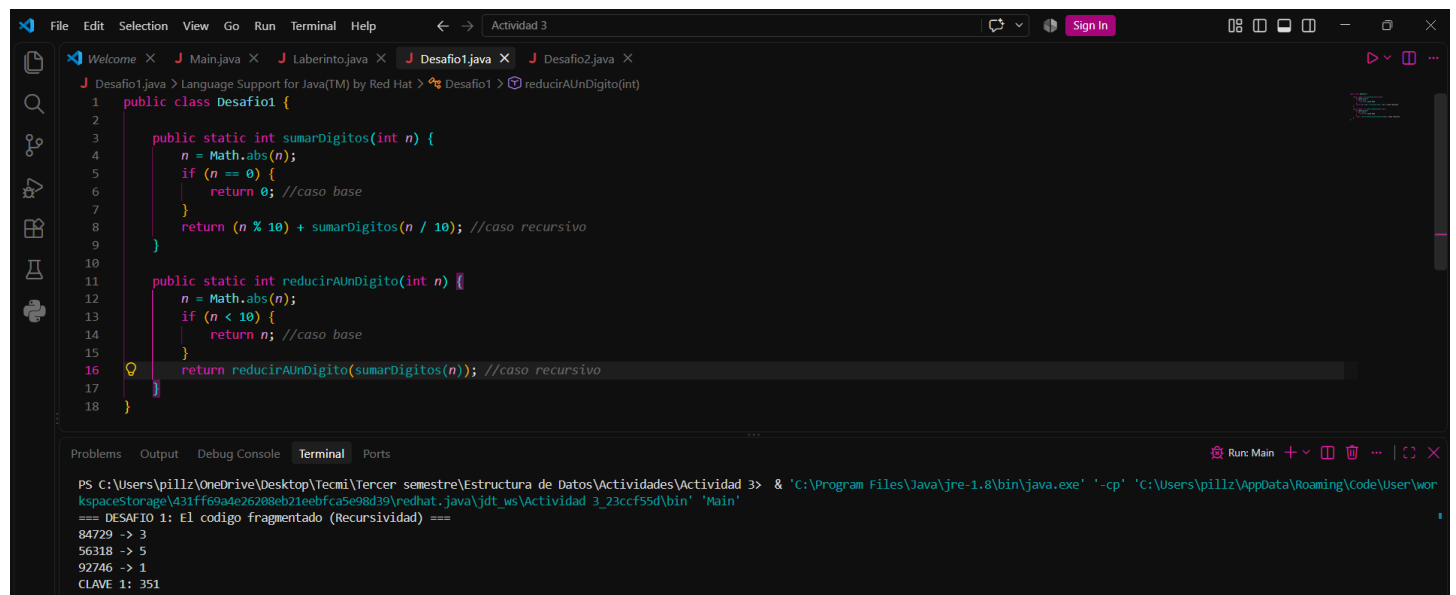


Escape del Laboratorio

Actividad 3

Pruebas realizadas DESAFIO 1

Se comprobó que el algoritmo reduce correctamente cada bloque numerico hasta obtener un solo digito utilizando recursividad, ademas genera correctamente la Clave 1 a partir de los resultados obtenidos



```
public class Desafio1 {  
    1  
    2  
    3  
    4    public static int sumarDigitos(int n) {  
    5        n = Math.abs(n);  
    6        if (n == 0) {  
    7            return 0; //caso base  
    8        }  
    9        return (n % 10) + sumarDigitos(n / 10); //caso recursivo  
    10    }  
    11  
    12    public static int reducirAUnDigito(int n) {  
    13        n = Math.abs(n);  
    14        if (n < 10) {  
    15            return n; //caso base  
    16        }  
    17        return reducirAUnDigito(sumarDigitos(n)); //caso recursivo  
    18    }  
}
```

```
PS C:\Users\pillz\OneDrive\Desktop\Tecmi\Tercer semestre\Estructura de Datos\Actividades\Actividad 3> & 'C:\Program Files\Java\jre-1.8\bin\java.exe' '-cp' 'C:\Users\pillz\AppData\Roaming\Code\User\workspaceStorage\431ff69a4e26208eb21eebfca5e98d39\redhat.java\jdk_ws\Actividad_3_23ccf55d\bin' 'Main'  
=== DESAFIO 1: El codigo fragmentado (Recursividad) ===  
84729 -> 3  
56318 -> 5  
92746 -> 1  
CLAVE 1: 351
```

Pruebas realizadas DESAFIO2

Se comprobó que el algoritmo encuentra correctamente el valor maximo del arreglo utilizando la estrategia de divide y venceras, sin ordenar el arreglo ni recorrerlo con un ciclo tradicional

The screenshot shows an IDE with the following components:

- Editor:** Contains a Java file named `Desafio2.java` with the following code:

```
1 public class Desafio2 {  
2  
3     public static int maximoDivideyVenceras(int[] datos) {  
4         if (datos == null || datos.length == 0) {  
5             throw new IllegalArgumentException("El arreglo no puede estar vacío");  
6         }  
7         return maximoRec(datos, inicio: 0, datos.length - 1);  
8     }  
9 }
```
- Terminal:** Displays the output of the program execution:

```
=== DESAFIO 2: El radar de emergencia (Divide y Venceras) ===  
Valor máximo encontrado: 947  
CLAVE 2: 947  
  
=== ACCESO AL SISTEMA ===  
Contraseña generada: 351-947  
  
=== DESAFIO 3: La ruta de escape (Backtracking) ===  
0,0  
0,1  
0,2  
1,2  
2,2  
2,1  
2,0  
3,0  
4,0  
4,1  
4,2  
4,3  
4,4  
2,3  
2,4  
1,4  
0,4  
3,4  
4,4  
  
RUTA:0,0;0,1;0,2;1,2;2,2;2,3;2,4;3,4;4,4
```

Pruebas realizadas LABERINTO

Se realizaron pruebas para verificar el comportamiento del programa en diferentes escenarios.

Caso 1. Ruta directa

Se comprobó que el algoritmo encuentra correctamente una ruta sencilla desde el inicio hasta la salida

The screenshot shows a terminal window with the following output:

```
--- CASO 1 - Ruta directa ---  
  
0,0  
0,1  
0,2  
0,3  
  
RUTA:0,0;0,1;0,2;0,3  
  
METRICAS:retrocesos=4;estados=4
```

Caso 2. Callejón sin salida

Se verificó que el algoritmo retrocede cuando un camino termina en un obstáculo y continúa explorando otras opciones

```
--- CASO 2 - Callejon sin salida ---  
0,0  
1,0  
2,0  
0,1  
0,2  
1,2  
2,2  
  
RUTA:0,0;0,1;0,2;1,2;2,2  
  
METRICAS:retrocesos=7;estados=7
```

Caso 3. Varias rutas posibles

Se comprobó que el algoritmo explora diferentes caminos y conserva la ruta más corta

```
--- CASO 3 - Varias rutas ---
0,0
1,0
2,0
3,0
4,0
4,1
4,2
4,3
4,4
2,1
2,2
2,3
2,4
1,4
0,4
3,4
4,4
0,1
0,2
0,3
0,4
1,4
2,4
3,4
4,4
2,3
2,2

RUTA:0,0;1,0;2,0;3,0;4,0;4,1;4,2;4,3;4,4

METRICAS:retrocesos=27;estados=27
```

Caso 4. La primera ruta no es la más corta

Se verificó que el algoritmo continúa explorando después de encontrar una solución para determinar si existe una ruta mejor

```
--- CASO 4 - Primera ruta no es la mas corta ---  
0,0  
1,0  
2,0  
2,1  
2,2  
2,3  
2,4  
1,4  
0,4  
0,1  
0,2  
0,3  
0,4  
  
RUTA:0,0;0,1;0,2;0,3;0,4  
  
METRICAS:retrocesos=13;estados=13
```

Caso 5. Sin solución

Se comprobó que el programa identifica correctamente un laberinto donde no existe una ruta hacia la salida

```
--- CASO 5 - Sin solucion ---  
0,0  
0,1  
0,2  
  
RUTA: NO EXISTE SOLUCION  
  
METRICAS:retrocesos=3;estados=3
```

Conclusión

Esta actividad permitió aplicar tres técnicas fundamentales de programación para resolver problemas con diferentes estrategias. La recursividad se utilizó para reducir un problema hasta un caso base, divide y vencerás permitió resolver un problema mediante la combinación de subproblemas y el backtracking hizo posible explorar diferentes rutas para encontrar la solución más corta en un laberinto